# Tribhuvan University

## Faculty of Humanities and Social Sciences

**A Project Report on**

TRAVEL & ITINERARY MANAGEMENT SYSTEM

## Submitted to:

Department of Computer Application

Nepal Kasthamandap College, Kalanki

***In partial fulfillment of the requirements for the Bachelor in Computer Application***

## Submitted by:


**Pranish Kumar Ranjit [Reg no 6-2-144-13-2022]**

**Sagar Nepal [Registration No.: 6-2-144-18-2022]**


**May , 2026**

**Under the supervision of**

**Er. Achyut Shah Thakuri**

# Tribhuvan University

## Faculty of Humanities and Social Sciences

## Nepal Kasthamandap College

Kalanki, Kathmandu

Bachelor in Computer Applications (BCA)

# Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by **Pranish Kumar Ranjit** and **Sagar Nepal** entitled **"Travel & Itinerary Management System"** in the Partial Fulfillment of requirement for the degree of Bachelor in Computer Application is recommended for that final evaluation.

_____

Er. Achyut Shah Thakuri
Project Supervisor
BCA Department
Nepal Kasthamandap College

# Tribhuvan University
## Faculty of Humanities and Social Sciences
## Nepal Kasthamandap College
Kalanki, Kathmandu

Bachelor in Computer Applications (BCA)

# Letter of Approval

This is certified that this project prepared by **Pranish Kumar Ranjit** and **Sagar Nepal** entitled **"Travel & Itinerary Management System"** in the Partial Fulfillment of requirement for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Er. Achyut Shah Thakuri
Supervisor
BCA Department
Nepal Kasthamandap College

Er. Sujan Poudel
Program Coordinator
Nepal Kasthamandap College

Mr. Roshan Poudel
Internal Examiner
BCA Department

Mr. Ramesh Singh Saud
External Examiner
Padma Kanya Multiple Campus

# Abstract

**The Travel & Itinerary Management System** is an innovative web-based application designed to simplify and enhance travel planning within selected cities, namely Kathmandu, Pokhara, and Biratnagar. The primary objective of this project is to provide a structured and user-friendly platform where users can explore hotels, restaurants, and tourist attractions, and create personalized travel itineraries efficiently. The system replaces traditional manual planning methods with a well-organized digital solution supported by a Database Management System (DBMS), ensuring better data management and accessibility. All travel-related information, including pricing and location details, is controlled by an administrator to maintain accuracy and consistency. Additionally, the system integrates the 0/1 Knapsack algorithm to recommend an optimal selection of places within a user-defined budget, thereby enhancing decision-making and overall travel experience. With features such as map-based visualization, cost estimation, and secure user interaction, the platform improves planning efficiency while promoting a reliable, organized, and paperless approach to travel and itinerary management.

***Keywords:*** *Travel Planner, Itinerary Management, Knapsack Algorithm, Map Integration*

# Acknowledgement

We would like to express our special thanks to our supervisor **Er. Achyut Shah Thakuri** who gave us the golden opportunity to do this wonderful project on the topic of **The Travel & Itinerary Management System** and we came to know about so many new things, new tools and technologies. We are especially grateful for his unwavering support and guidance, which greatly contributed to our personal development. We would also like to thank our parents and friends who helped us a lot in finalizing this project within the limited time frame. Finally, we are grateful to them who have directly or indirectly helped us during this project.

Yours Sincerely,

**Pranish Kumar Ranjit**

**Sagar Nepal**

# Table of Contents

**Supervisor's Recommendation** ..... **i**
**Letter of Approval** ..... **ii**
**Abstract** ..... **iii**
**Acknowledgement** ..... **iv**
**List of Figures** ..... **vii**
**List of Tables** ..... **viii**
**List of Listings** ..... **ix**
**List of Abbreviations** ..... **x**
**Chapter 1: Introduction** ..... **1**
1.1 Introduction ..... 1
1.2 Problem statement ..... 2
1.3 Objectives ..... 2
1.4 Scope and Limitation ..... 2
1.4.1 Scope ..... 2
1.4.2 Limitation ..... 3
1.5 Development Methodology ..... 3
1.6 Report Organization ..... 4
**Chapter 2: Background Study and Literature Reviews** ..... **5**
2.1 Background Study ..... 5
2.2 Literature Review ..... 7
**Chapter 3: System Analysis and Design** ..... **9**
3.1 System Analysis ..... 9
3.1.1 Requirement Analysis ..... 9
3.1.2 Feasibility Study ..... 11
3.1.3 Data Modeling ..... 12
3.1.4 Process Modeling (DFD) ..... 13
3.2 System Design ..... 15
3.2.1 Gantt Chart ..... 15
3.2.2 Architectural Design ..... 16
3.2.3 Flow Chart ..... 17
3.2.4 Database Schema Design ..... 20
3.2.5 Wireframing ..... 21

3.2.6 Interface Design ..... 22
3.2.7 Algorithm ..... 23
**Chapter 4: Implementation and Testing ..... 26**
4.1 Implementation ..... 26
4.1.1 Tools Used ..... 26
4.1.2 Implementation Details of Modules ..... 28
4.2 Testing ..... 30
4.2.1 Test Cases for Unit Testing ..... 30
4.2.2 Test Cases for System Testing ..... 32
4.3 Result and Analysis ..... 33
**Chapter 5: Conclusion and Future Recommendations ..... 35**
5.1 Lesson Learnt ..... 35
5.2 Conclusion ..... 36
5.3 Future Recommendation ..... 36
**References**
**Appendices**

# List of Figures

Figure 1.1 Agile Methodology.....3
Figure 3.1 Use Case Diagram for Travel and Itinerary Management System.....10
Figure 3.2 ER Diagram for Travel and Itinerary Management System.....12
Figure 3.3 DFD level 0 for Ticket and Itinerary Management System .....13
Figure 3.4 DFD level 1 for Ticket and Itinerary Management System .....14
Figure 3.5Architecture Design.....17
Figure 3.6 User Flowchart for Travel and Itinerary Management System .....18
Figure 3.7 Admin Flowchart for Travel and Itinerary Management System .....19
Figure 3.8 Database Schema Design for Travel and Itinerary Management System .....20
Figure 3.9 Wire Framing of Explore Page.....21
Figure 3.10 Wire Framing of Login Page.....21
Figure 3.11 Wire Framing of Auto-itinerary Generation Page.....22
Figure 3.12 Interface Design of Explore Page.....22
Figure 3.13 Interface Design of Login Page.....22
Figure 3.14 Interface Design of Auto-itinerary Generation Page.....23

# List of Tables

Table 3.1 Project schedule .....16
Table 4.1 Test Case for Login.....30
Table 4.2 Test Case for Itinerary Generation.....31
Table 4.3 Test Case for Admin Pannel .....31
Table 4.4 System Testing for Travel and Itinerary Management System .....32

# List of Listings

Listing 4.1 Code Snippet to add new places .....28
Listing 4.2 Code Snippet to get all places.....28
Listing 4.3 Code Snippet to Update places .....29
Listing 4.4 Code Snippet to Delete places .....29
Listing 4.5 Code Snippet of Algorithm Implementation .....29

# List of Abbreviations

The following table describes the significance of various abbreviations and acronyms used throughout the report.

| Abbreviation | Meaning |
|---|---|
| CSS | Cascade Style sheet |
| DFD | Data Flow Diagram |
| JS | JavaScript |
| MySQL | My Structured Query Language |
| TIMS | Travel & Itinerary Management System |

# Chapter 1: Introduction

## 1.1 Introduction

In today's busy world, travelers often find themselves overwhelmed by the need to visit multiple websites, applications, and tools to research destinations, book services, and arrange schedules. This fragmented process not only consumes significant time but also increases complexity, making trip planning stressful and inefficient. To overcome these challenges, the proposed system provides a unified and structured solution that integrates all essential travel planning features into a single, user-friendly interface.

Through this platform, users can explore destinations, discover hotels, restaurants, and tourist attractions, and create personalized itineraries based on their preferences and budget. The system focuses on selected cities, namely Kathmandu, Pokhara, and Biratnagar, where all travel-related data is carefully managed by an administrator to ensure accuracy, consistency, and reliability. This curated approach enhances user trust and improves overall service quality. A key feature of the system is its ability to provide cost estimation and budget-based recommendations. By utilizing structured data stored in a Database Management System (DBMS), the platform enables users to calculate their total travel expenses efficiently. Furthermore, the integration of the 0/1 Knapsack algorithm allows the system to recommend an optimal combination of places within a user-defined budget, thereby supporting better decision-making and maximizing user satisfaction.

In addition to planning, the system incorporates map-based visualization using geographical coordinates, allowing users to view locations interactively and understand distances between places. This enhances the overall usability and provides a more realistic representation of travel routes. The platform also ensures secure user authentication, efficient data handling, and a responsive interface, making it accessible and reliable for a wide range of users.

Overall, the Travel and Itinerary Management System aims to transform traditional travel planning into a streamlined, efficient, and enjoyable digital experience. By reducing manual effort, minimizing confusion, and promoting a paperless environment, the system empowers users to plan their journeys with greater ease, confidence, and precision which enhances the overall usability and provides a more realistic representation of travel routes. The platform also ensures secure user authentication, efficient data handling, and a responsive interface, making it accessible and reliable for a wide range of users and maximize user satisfaction.

## 1.2 Problem statement

Planning a trip can be a difficult task for many travelers. Coordinating various aspects like selecting destinations and organizing itineraries can quickly become overwhelming. Most current travel solutions aren't well-integrated, leading to fragmented and inefficient experiences. Travelers often find themselves hopping between multiple platforms just to research destinations and keep track of their plans. This scattered approach is not only time consuming but also frustrating. This project aims to tackle these challenges by offering a unified travel and itinerary management system. This platform is designed to simplify the entire travel planning process. With a built-in interface that brings together all essential travel services, we aim to provide a seamless and efficient experience for users. By consolidating all travel-related tasks into one easy-to-use system, we hope to make planning and enjoying trips much more convenient and enjoyable for everyone. Ultimately, the system enhances user satisfaction by saving time and reducing stress. It ensures that travelers can focus more on creating memories rather than worrying about logistics. Additionally, the system incorporates budget optimization techniques to help users make cost-effective decisions while maximizing their travel experience.

## 1.3 Objectives

The project aims to meet the following objectives:

- To create itinerary creation system which offer a user-friendly interface with detailed descriptions, ratings, to help travelers make informed decisions.
- To provide the expense tracking and discover attractions and destinations of the proposed itinerary.
- To provide the hub for itinerary creation using auto-itinerary creating system.

## 1.4 Scope and Limitation

### 1.4.1 Scope

Previously, handling travel planned manually was tedious and time-consuming, with a higher probability of errors. With the increasing demand for convenience and efficiency, travel and itinerary management system offers an automatic interface for managing all aspects of travel planning services from anywhere, at any time. The web application will be designed to provide users with a seamless travel experience by incorporating robust features such as destination discovery and detailed information retrieval.

### 1.4.2 Limitation

- The system won't have google Maps APIs as it is not feasible because of its cost issues.
- Some travel places might have less or outdated information.
- High cost and limited resources may stop adding advanced features.

## 1.5 Development Methodology

Travel and Itinerary Management System is built using Agile technique. Agile methodology's primary goal is to manage and deliver projects in a collaborative, flexible, iterative manner with a strong importance on customer satisfaction and ongoing improvement. Agile places a strong attention on flexibility and change responsiveness to overcome the faults of traditional project management techniques.

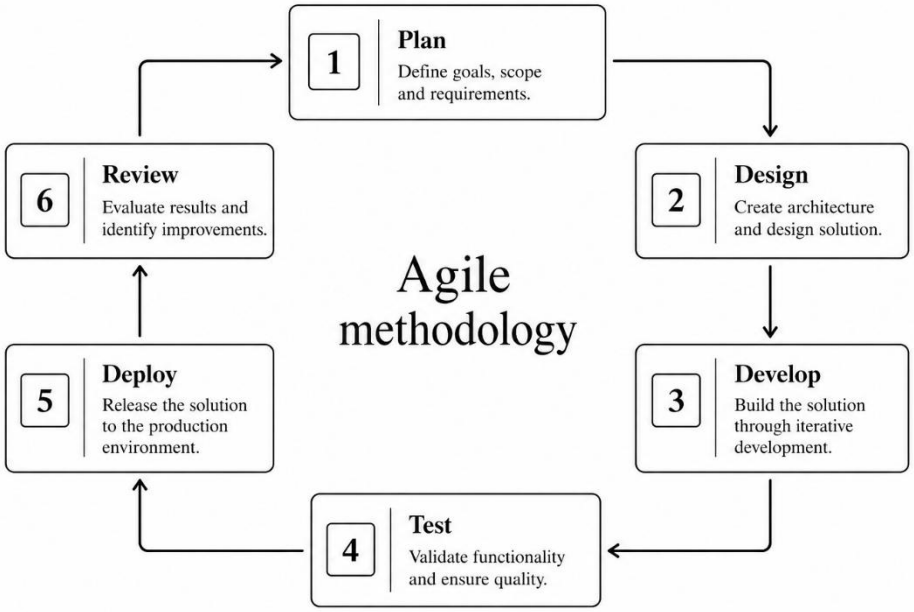



**Figure 1.1 Agile Methodology**

Agile Methodology is followed by the design of the software. First, the system will be made conceptual by making a proper plan and will be developed accordingly. After developing a project testing and deployment of the software will be conducted. The main thing which convinced me to use agile methodology in my system is, my system needs a flexible or changing environment to get it deployed starting from the planning phase. I can make changes according to the needs from time to time and agile provides flexibility to changing development environments and requirements. Also, agile allows us continuous integration and testing to assure quality throughout the development process.

## 1.6 Report Organization

The report is organized as follow:

**Chapter 1:** Introduction of the project along with the problem statement, objectives, scope, limitation and development methodology.

**Chapter 2:** Background study related to the project along with general descriptions of project functions and components. Literature review in order to have broader understanding of the project concepts based on research done previously and analyze similar systems for comparison with projects.

**Chapter 3:** System Analysis and design of the system using various charts and figures. Functional requirements using use cases and other techniques. Database schema, interface design and deployment diagram.

**Chapter 4:** Tools and techniques used for project implementation along with algorithms used in the project and creations of test cases to test the system as unit and as a whole.

**Chapter 5:** Lesson learnt from start to finishing the project, future recommendations for other projects and project conclusions.

# Chapter 2: Background Study and Literature Reviews

## 2.1 Background Study

In today's world, planning a trip can be complex and time-consuming, often involving multiple platforms for finding information, booking, and organizing schedules. This system aims to simplify this process by offering a single solution. It's designed to make travel planning from exploring destinations to customizing itineraries and booking accommodation easier and more straightforward. By integrating all these essential travel services into one interface, our system strives to provide travelers with a seamless experience, ensuring convenience and efficiency throughout their journey. By end of the project the users will be able to create and manage detailed trip itineraries with activities and notes and get personalized recommendations for destinations, attractions, and restaurants.

GetYourGuide is a global online platform that enables travelers to discover, compare, and book tours, activities, and experiences in destinations around the world. Founded in 2008 and headquartered in Berlin, it partners with local guides, tour operators, and attractions to offer a wide range of options, from guided city tours and cultural experiences to skip-the-line tickets for popular landmarks. The platform is designed to be user-friendly, providing detailed descriptions, customer reviews, and secure booking, making it easy for travelers to plan and enjoy activities in advance. It is especially useful for those who want to organize their trips efficiently and access unique local experiences. [1]

Zip Zap Travels is a travel agency offering personalized itineraries, exclusive deals, and 24/7 support to ensure seamless and memorable trips. They provide a variety of travel options, including flights, hotels, tours, and visa assistance, catering to diverse travel needs. The agency is known for its commitment to creating exceptional travel experiences, with a focus on customer satisfaction and attention to detail. Their user-friendly platform allows travelers to easily book their journeys, making travel more accessible. [2]

Nepal Trip Planner Pvt. Ltd. is a Kathmandu-based tour operator specializing in customized travel experiences across Nepal, Bhutan, India, and Tibet. They offer a diverse range of services, including private tours, trekking expeditions, cultural and religious journeys, and luxury packages. Their offerings encompass popular destinations such as Everest Base Camp, Annapurna, Mukti Nath, and Bhutan's cultural heritage sites. The company emphasizes personalized itineraries tailored to individual preferences and budgets, ensuring a unique and memorable travel experience. [3]

Tripadvisor is a leading global travel platform that empowers users to plan and book their ideal trips by providing a vast array of travel-related content and services. Founded in 2000 by Stephen Kaufer and Langley Steinert, the company is headquartered in Needham, Massachusetts. Tripadvisor operates in over 40 countries and 20 languages, featuring approximately 1 billion reviews and opinions on around 8 million establishments, including accommodations, restaurants, experiences, airlines, and cruises. The company continues to innovate and adapt to the evolving travel industry, aiming to be the world's most trusted source for travel and experiences. [4]

Nepal Trekking Planner Pvt. Ltd. is a Kathmandu-based, family-owned trekking agency specializing in personalized adventures across Nepal, Bhutan, Tibet, and India. Established in 2000, the company offers a wide range of services, including trekking, peak climbing, cultural tours, helicopter tours, and wildlife safaris. Their offerings cater to various interests, from challenging treks like the Everest Base Camp and Annapurna Circuit to cultural explorations and family-friendly holidays. The company emphasizes responsible tourism and community support, ensuring that their operations benefit local communities. With over 10,000 travelers joining their tours annually, Nepal Trekking Planner has built a reputation for delivering exceptional experiences. [5]

Viator is a leading global online marketplace for tours, activities, and attractions, owned by Tripadvisor. The platform offers travelers access to thousands of experiences worldwide, ranging from city tours and day trips to adventure activities and cultural excursions. With user-friendly search and booking features, Viator provides detailed descriptions, customer reviews, and secure payment options. Its partnership with local operators ensures diverse offerings, including skip-the-line tickets and private tours. As part of Tripadvisor, Viator benefits from a vast customer base and global reach, making it one of the most popular choices for booking travel experiences. [6]

Musement is a digital platform that allows travelers to book cultural activities, museum passes, tours, shows, and attractions in multiple destinations. Founded in Italy in 2013, the company quickly expanded its services across Europe and beyond. The platform focuses on making cultural and entertainment experiences more accessible, with curated listings and personalized recommendations. Since 2018, Musement has been part of TUI Group, further enhancing its global offerings. Musement integrates ticketing, reservations, itinerary support into one app, helping travelers save time and enjoy a smooth experience. Since 2018, Musement has been part of TUI Group, further enhancing global offerings and support in to one app. [7]

Trekiz is an online travel platform specializing in customized and multi-destination itineraries. It provides tools that allow users to plan trips by combining flights, hotels, tours, and activities into a single package. Based in China, Trekiz partners with both international and local service providers to offer a wide variety of travel options. The platform is designed to give travelers more flexibility and control over their journeys, catering especially to those who prefer tailor-made experiences rather than fixed packages. By consolidating multiple services, Trekiz helps streamline the often-complex process of trip planning. [8]

Access Nepal Tour & Trekking is a Kathmandu-based travel agency founded with the mission of promoting sustainable and women-led tourism in Nepal. The company specializes in trekking, adventure tours, cultural explorations, and spiritual journeys across Nepal, Bhutan, and Tibet. With experienced local guides and a focus on safety, Access Nepal provides personalized itineraries tailored to different fitness levels and preferences. Their offerings range from classic treks like Everest Base Camp to cultural tours and short excursions, ensuring a broad appeal. The agency also emphasizes eco-friendly practices and community involvement, making it a responsible choice for travelers. [9]

Travel Local is a unique platform that connects travelers with local experts and tour operators around the world. Instead of relying on large international agencies, users can directly plan and book trips with on-the-ground professionals who know the destinations best. Travel Local's model ensures more authentic, customized, and often more affordable travel experiences. For Nepal and surrounding regions, the platform partners with local operators to provide trekking, cultural tours, wildlife safaris, and heritage trips. By empowering local businesses and offering travelers personalized service, Travel Local combines convenience with authenticity, making it an increasingly popular choice for meaningful travel. [10]

## 2.2 Literature Review

The travel and tourism industry has seen significant advancements due to the integration of digital technologies. Before beginning the project, I conducted research on existing Travel Management Systems. Here are some of the examples that I found from the research. These existing systems provided valuable insights into current features, functionalities, and limitations in travel planning platforms. The analysis helped in identifying gaps and opportunities, which guided development of this proposed system.

An innovative online tourist management system has been introduced to address the inefficiencies in the travel industry. Utilizing HTML, PHP, and Microsoft SQL Server, this system centralizes booking and information transmission, thereby improving customer experience and corporate productivity. The study highlights the system's potential to streamline vacation planning, reduce errors, and make the travel process more enjoyable and accessible for users. [11]

A longitudinal study from 2007 to 2012 reveals significant changes in how American travelers use the Internet. It identifies a bifurcation between traditional online travelers and those seeking deeper, more authentic experiences through alternative digital channels. This study highlights the growing importance of personalized and experience-driven travel planning in the digital age. It shows that users increasingly rely on integrated platforms that offer convenience along with meaningful recommendations. These findings emphasize the need for systems that combine functionality, personalization, and efficiency, which has influenced the design of this project. [12]

This shift underscores the importance for travel businesses to adapt to evolving consumer behaviors to stay competitive Application Quality of Service. A study explores the use of IoT technology to enhance self-guided tours. The proposed system creates personalized travel itineraries based on user preferences and professional recommendations, optimizing routes using branch-and-bound algorithm. [13]

This system significantly improves the quality and efficiency of self-guided tours by automating the itinerary planning process. A personal navigation system (PNS) has been developed to help tourists navigate multiple destinations efficiently. By using a genetic algorithm, the PNS calculates optimal routes in real time based on user preferences and specified timings. Accessible via portable devices, [14]

Building on IoT advancements, another study focuses on RFID technology for passive sensing in path suggestion systems. It integrates user emotions and product similarities into an improved ant colony algorithm, enhanced by a particle swarm algorithm. This model, tested on real-world data, offers highly accurate and efficient tourist route recommendations, demonstrating significant improvements over traditional methods. [15]

# Chapter 3: System Analysis and Design

## 3.1 System Analysis

System analysis is a review of a technological system, like a software package, for troubleshooting, development or improvement purposes. Finding areas for improvement, making sure the system accomplishes its goals, and making suggestions for improvement are the main objectives of system analysis. With a systems analysis, considering the goals of the system is important for solving problems and creating efficiencies. From there, dividing a system into components can make it easier to perform individual analysis that influences the complete system.

### 3.1.1 Requirement Analysis

Requirement analysis is a crucial stage that involves identifying, recording, and evaluating the needs and expectations of users to make sure the report addresses the relevant topics and offers insightful information. The goal of the requirement study for Travel and Itinerary Management System is to identify the critical features and functionalities required to meet user challenges. It also ensures that the system is designed in a user-centric manner, aligning technical solutions with real-world travel planning needs and expectations.
This system needs to fulfill the following functional and non-functional requirements.

**i. Functional Requirements**

- Customers must be able to create and manage detailed trip itineraries with activities and notes.
- Customers must be able to get personal recommendations for destinations, attractions, and restaurants.
- The customers must be able to share travel plans through links or email.
- The customers must be able to track and categorize travel expenses.

A use case diagram is a type of diagram in UML (Unified Modeling Language) that visually represents the functional requirements of a system by showing the interactions between external users, known as actors, and the system itself. It illustrates what actions or tasks (use cases) the system performs from the user's perspective, helping to clearly define system behavior, scope, and user involvement. Use case diagrams are mainly used during the requirement analysis phase to provide a simple and understandable overview of how different users interact with the system.

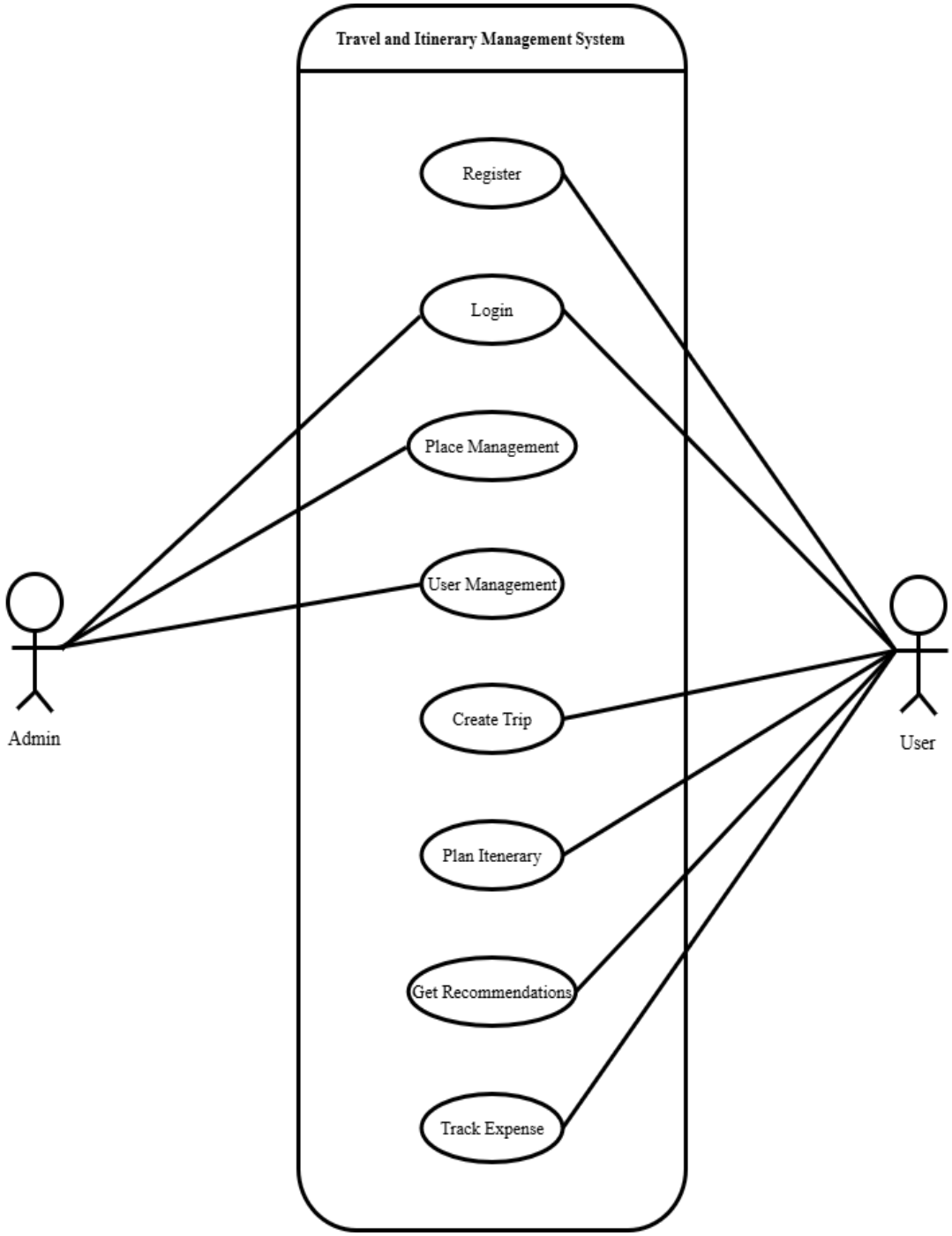



**Figure 3.1 Use Case Diagram for Travel and Itinerary Management System**

The above Use Case Diagram shows how different actors interact with the system. The main actor is the User, who can register/login, create trips, plan itineraries, get recommendations, track expenses, share itineraries, and export them to PDF. The admin is another actor who manages users and updates travel-related data (such as destinations and trip information). The diagram highlights the relationship between these actors and the system's main functions, showing how the application supports both travelers and administrators in managing trips effectively.

**ii. Non-Functional Requirements**

Different non-functional requirements have been studied and identified and are listed below:

- Security: Data should be accessible only by authorized users to ensure privacy and protection.
- Performance: The system should respond quickly and handle transactions efficiently.
- User Interface: The interface should be simple, intuitive, and easy to navigate.
- Compatibility: The application should work on Android and iOS devices.
- Reliability: The system should be stable with minimal downtime and errors.

### 3.1.2 Feasibility Study

A feasibility study is a way to evaluate whether or not a project plan could be successful or not. Feasibility study includes consideration of all the possible ways to provide a solution to the given problem. The proposed solution should satisfy all the user requirements and should be flexible enough so that future changes can be easily done based on the future upcoming requirements.

**i. Technical feasibility**

This involves questions such as whether the technology needed for the system exists, how difficult it will be to build, and whether the firm has enough experience using that technology. The technology or tools which will be used are React.js and MYSQL for the database. So, the system is technically feasible.

**ii. Operational feasibility**

Operational Feasibility is the measure of how well a proposed system solves the problems and takes advantage of the opportunities identified during the scope definition. It includes everyone who creates, operates, or uses the system. Programs that reduce costs without reducing the quality of a product are an example of operational feasibility. The system will also need to be maintained and updated regularly to ensure it remains current and relevant.

**iii. Economic feasibility**

Economic feasibility is the most frequently used method for evaluating the effectiveness of a new system. This procedure is to determine the benefits and savings that are expected from the system and compare them with costs. We have estimated that the cost of the development of this system will be minimal which will benefit both users and developers.

### 3.1.3 Data Modeling

A database system's entities, relationships, and characteristics are represented graphically in an entity-relationship (ER) diagram. It facilitates the logical links and database structure 8 visualizations. An ER diagram gives a clear grasp of the database schema and shows how entities link to one another using symbols like lines, diamonds, and rectangles.

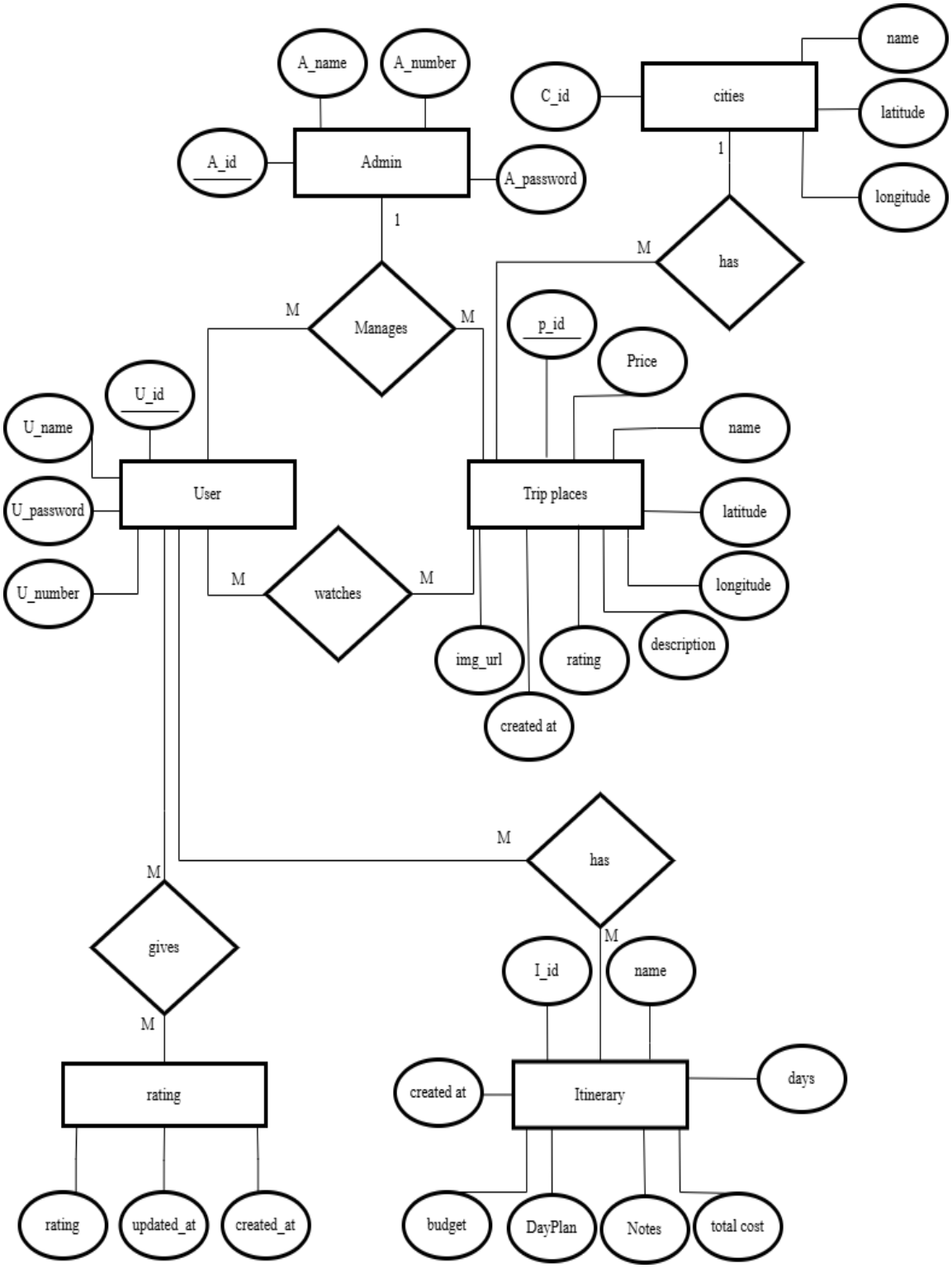



**Figure 3.2 ER Diagram for Travel and Itinerary Management System**

The Entity Relationship (ER) Diagram of the Travel & Itinerary Management System represents the structure of the database by showing the main entities users, cities, places, and place_ratings along with their attributes and relationships. The users entity stores user information, while cities contain details of supported locations. The places entity includes hotels, restaurants, and attractions, each linked to a specific city through a one-to-many relationship. The place_ratings entity connects users and places, allowing users to rate different places, forming a many-to-many relationship. Overall, the ER diagram ensures proper data organization, integrity, and efficient interaction between different components of the system.

### 3.1.4 Process Modeling (DFD)

A data flow diagram shows the flow of data through an information system graphically. It can show the flow of data that is saved, sent out, and received. The DFD makes no mention of the data flow within the system.

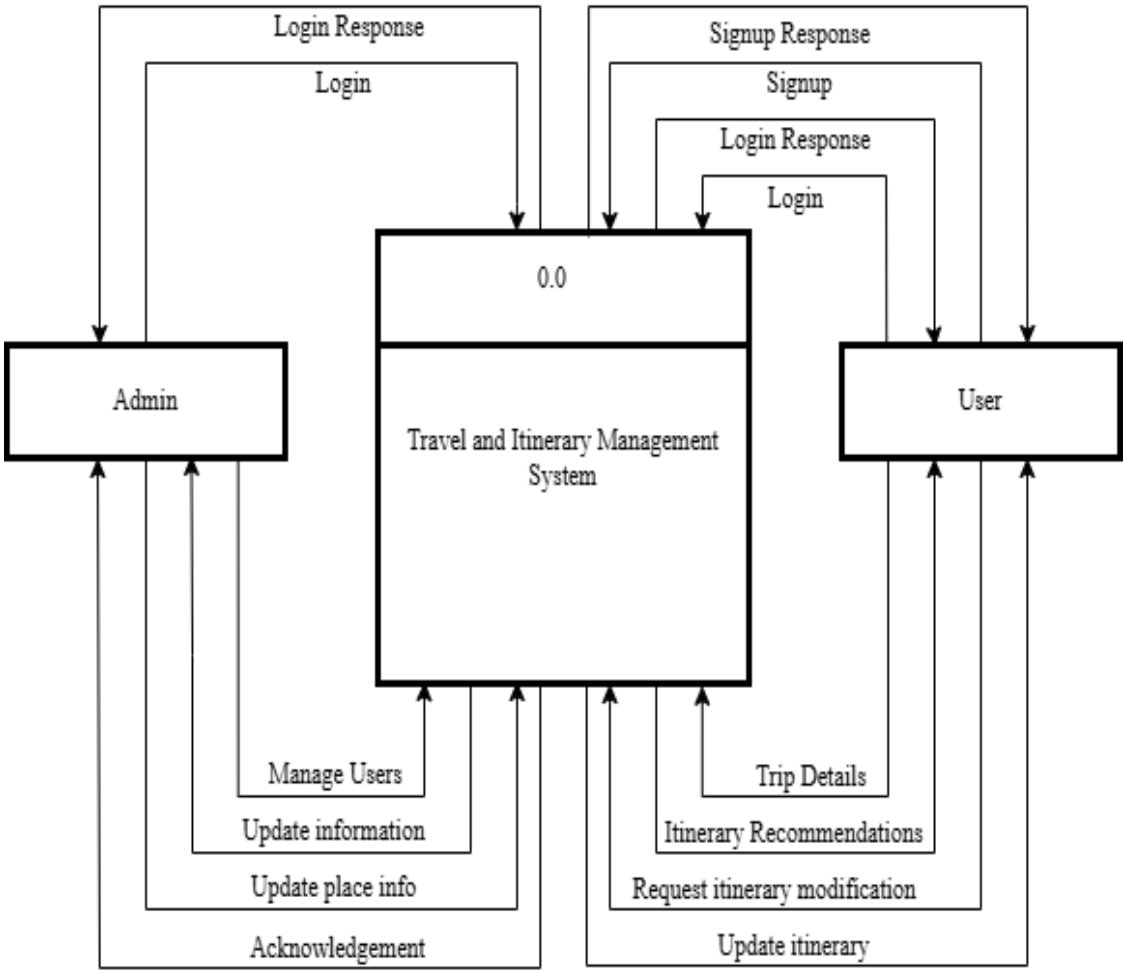



**Figure 3.3 DFD level 0 for Ticket and Itinerary Management System**

The Level 0 Data Flow Diagram (DFD) for the Travel Planning and Itinerary Management System shows the main interactions between external entities, the system, and the database. The system allows users to register, log in, create trips, add activities, and track expenses. Users also receive personalized itineraries, recommendations, and expense reports. The admin manages users and trips, while the system provides updated records and reports in return. Additionally, all trip-related data, including users, itineraries, and expenses, is stored and retrieved from the central database. This diagram gives an overview of how data flows in and out of the system, highlighting its core processes.

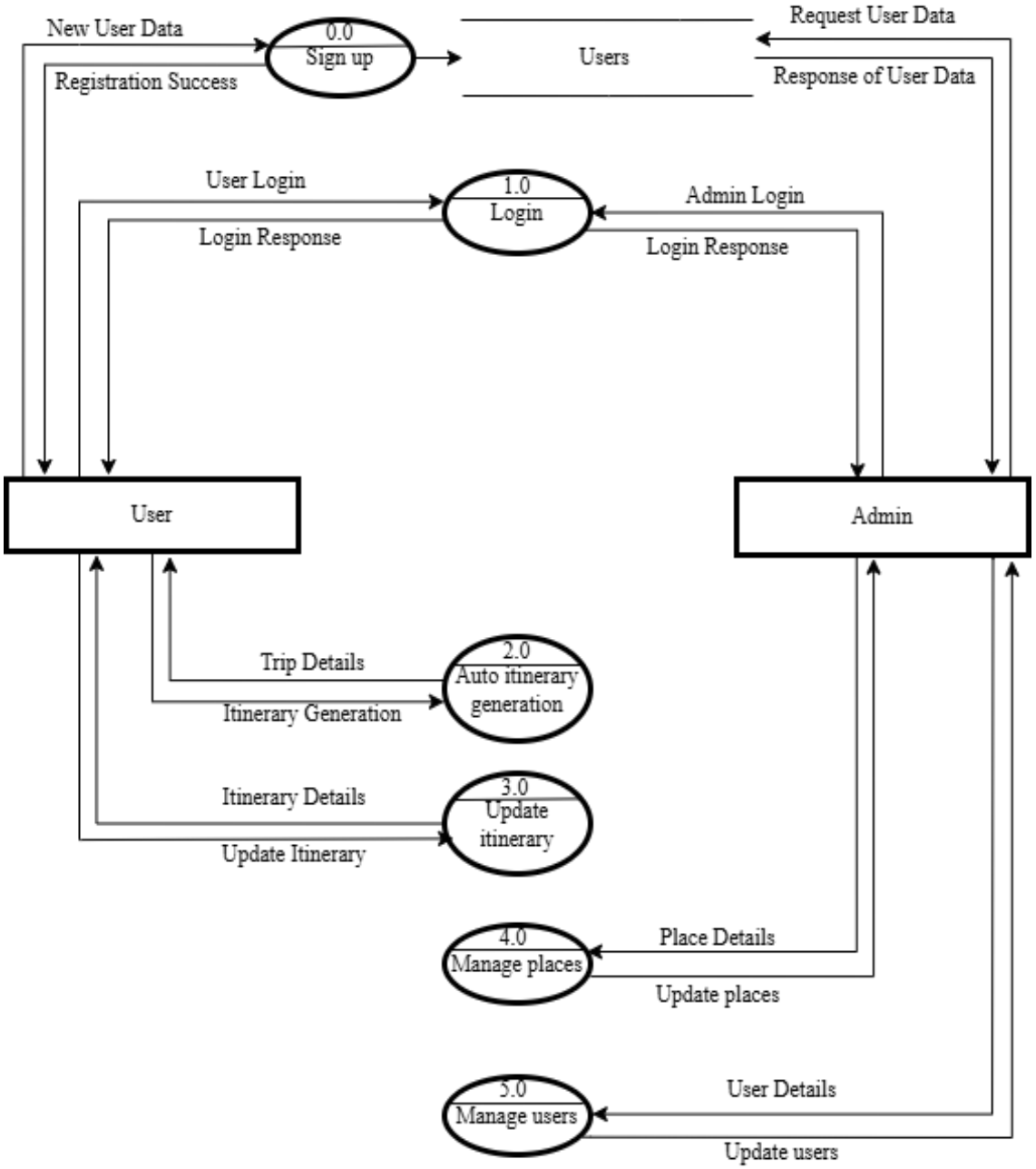



**Figure 3.4 DFD level 1 for Ticket and Itinerary Management System**

The Level 1 Data Flow Diagram (DFD) of the Travel & Itinerary Management System provides a detailed breakdown of the system's internal processes and how data flows between users, administrators, processes, and data stores. In this diagram, the User interacts with processes such as User Registration & Login (1.0), Explore Places (2.0), and Plan Trip (3.0) by providing inputs like login credentials, trip preferences, and place selections. The Admin interacts with the system through processes like Manage Places & Cities (4.0) and Rate Places (5.0) to control and update system data. The system stores and retrieves information from data stores including Users, Cities, Places, Ratings, and Trips, ensuring proper data management and consistency. Overall, the diagram illustrates how different modules work together to handle user requests, process data, and produce outputs such as trip plans, ratings, and updated system information.

## 3.2 System Design

System Design is a crucial phase in the system development life cycle that focuses on transforming the requirements identified during system analysis into a complete and structured technical solution. It involves defining the overall architecture of the system, including its components, modules, data structures, interfaces, and interactions between different parts of the system. During this phase, developers determine how the system will be built, how data will be stored and managed, and how different functionalities will be implemented efficiently. System Design also includes the selection of appropriate algorithms, database schema design, user interface layout, and system workflows to ensure that the final product meets performance, scalability, and usability requirements. Additionally, it incorporates planning tools such as Gantt charts to organize the development process and allocate tasks over time. By providing a clear blueprint for implementation, System Design ensures that the system is well-organized, reliable, and capable of fulfilling user needs effectively.

### 3.2.1 Gantt Chart

A Gantt chart is a visual aid for project management that shows tasks or activities on a horizontal time axis to show the project timeline. Its main purpose is to present a clean and well-organized view of the project schedule, indicating the beginning and ending dates, durations, and the recommended order of completion for each task. Project managers and team members can effectively manage resources, track progress, and identify dependencies between tasks by labeling tasks as bars on the chart. Gantt charts play a crucial role in project management by helping to plan, coordinate, and track activities.

**Table 3.1 Project schedule**

| Task/Date | 28th Oct | 17th Nov | 25th Dec | 6th Feb | 28th Feb |
|---|---|---|---|---|---|
| Planning | | | | | |
| Requirement gathering | | | | | |
| Sprint 1-Basic UI and DB Setup | | | | | |
| Testing Sprint 1 | | | | | |
| Sprint 2-Itinerary Module | | | | | |
| Testing Sprint 2 | | | | | |
| Sprint 3- Admin Panel | | | | | |
| Final Testing | | | | | |
| User Feedback and Improvements | | | | | |
| Documentation | | | | | |

The Gantt chart represents the project schedule for the Travel & Itinerary Management System, illustrating how different tasks were planned and executed over time. The project begins with the Planning phase in mid-September, followed by Requirement Gathering, which extends into October to define system. The first sprint is carried out in October, establishing the core structure of the application, followed by Testing Sprint 1. In November, the second sprint is developed, focusing on implementing key features such as trip generation and budget planning, followed by Testing Sprint 2 to ensure accuracy and performance. The third sprint is developed around early January, enabling management of cities, places, and user data. After development, Final Testing are conducted to ensure system stability before release. Throughout the entire timeline, Documentation is carried out continuously to record progress and system details.

### 3.2.2 Architectural Design

Architectural Design is a phase in system development where the overall structure of the system is planned by identifying its key components, such as frontend, backend, and database, and defining how these components communicate and work together.

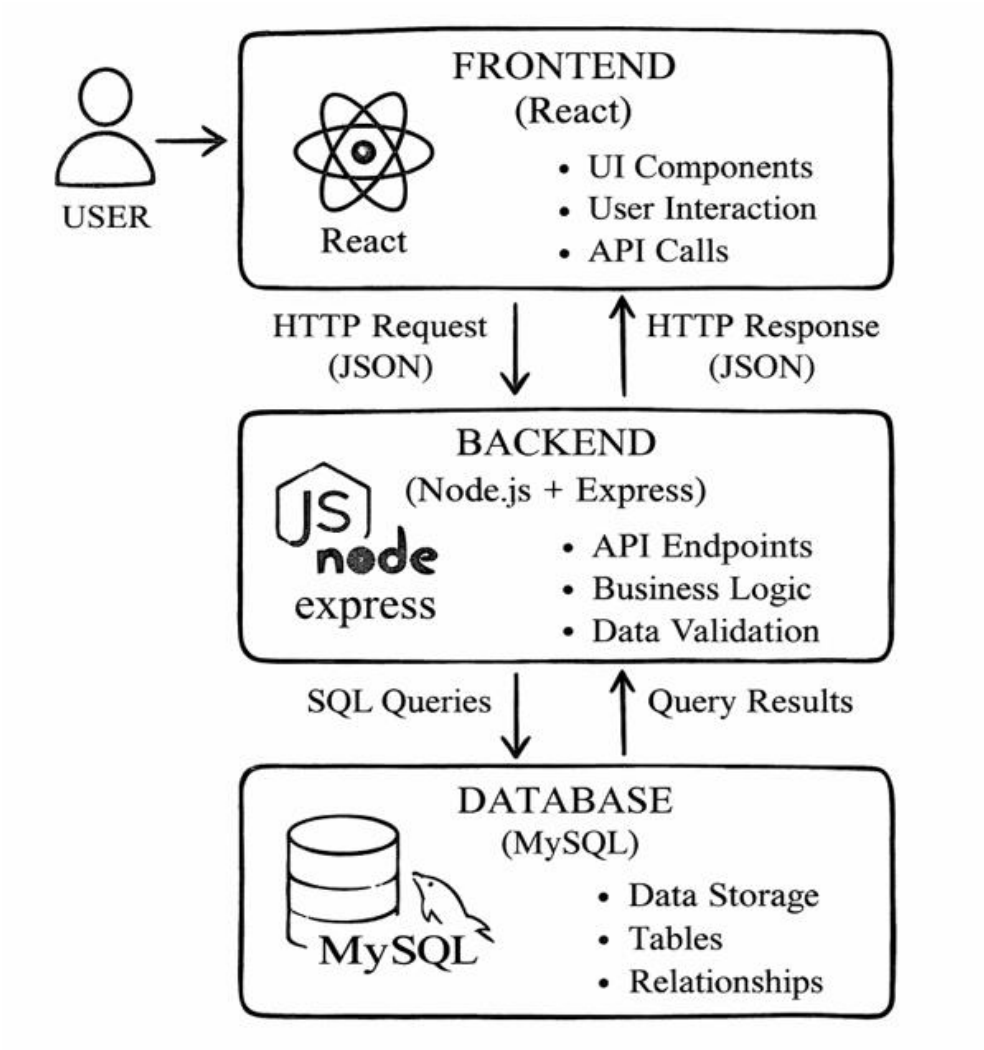



**Figure 3.5Architecture Design**

The system follows a three-tier architecture consisting of frontend, backend, and database layers. The frontend, built using React, handles user interaction and sends requests to the backend. The backend, developed using Node.js and Express, processes business logic and executes algorithms such as the 0/1 Knapsack for budget optimization. The backend communicates with the MySQL database to store and retrieve data. This layered architecture ensures scalability, maintainability, and efficient data flow within the system.

### 3.2.3 Flow Chart

A flowchart is a visual representation of a process, showing each step in chronological order. In the Personal Expense Tracker application, the flowchart illustrates how users and administrators interact with the system. Upon accessing the app, individuals are prompted to log in. The system verifies their credentials and, if invalid, redirects them to the registration page to provide the required information. Once logged in, the system identifies whether the person is a standard user or an administrator to assign the correct privileges.

### i. User Flowchart

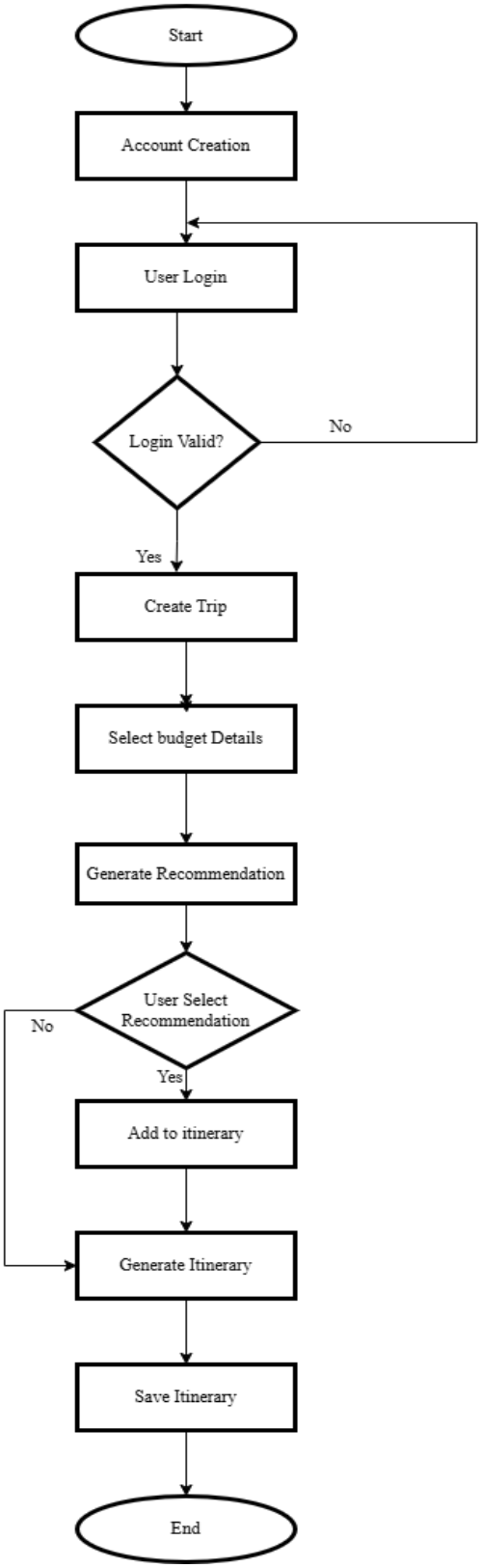



**Figure 3.6 User Flowchart for Travel and Itinerary Management System**

The user flowchart represents the sequence of actions a customer or end-user follows in the system. It starts with the login or registration step, where new users must create an account, while existing users log in using their credentials. After successful login, the user is directed to the main interface, where they can access available services such as searching, booking, or managing reservations. The flow includes options for browsing records, selecting preferences, and submitting requests. Once the required action is completed, the system processes the request and confirms the result. Finally, the user can either continue using the system or choose to logout, ending the session. This structured flow ensures a smooth and secure interaction for users while accessing the system's services.

**ii. Admin Flowchart**

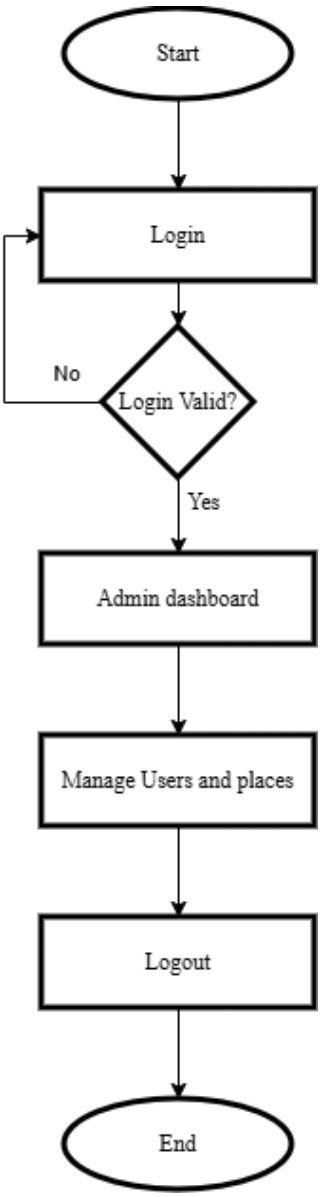



**Figure 3.7 Admin Flowchart for Travel and Itinerary Management System**

The admin flowchart illustrates the step-by-step process followed by the system administrator. It begins with the login step, where the admin must enter valid credentials to gain access. Once logged in, the admin is presented with the dashboard, which acts as the control center for managing the application. From here, the admin can perform multiple functions such as managing users, updating system records, monitoring activities, and handling reservations or data updates**.** The admin has the authority to add, update, or delete records as needed. After completing the required operations, the admin can choose to either continue working on the dashboard or logout from the system, which ends the session. This flow ensures that administrative tasks are properly secured, managed, and monitored within the system.

### 3.2.4 Database Schema Design

The Database Schema Design of the Travel & Itinerary Management System defines the structure of the database, including tables, attributes, and relationships required to store and manage data efficiently.

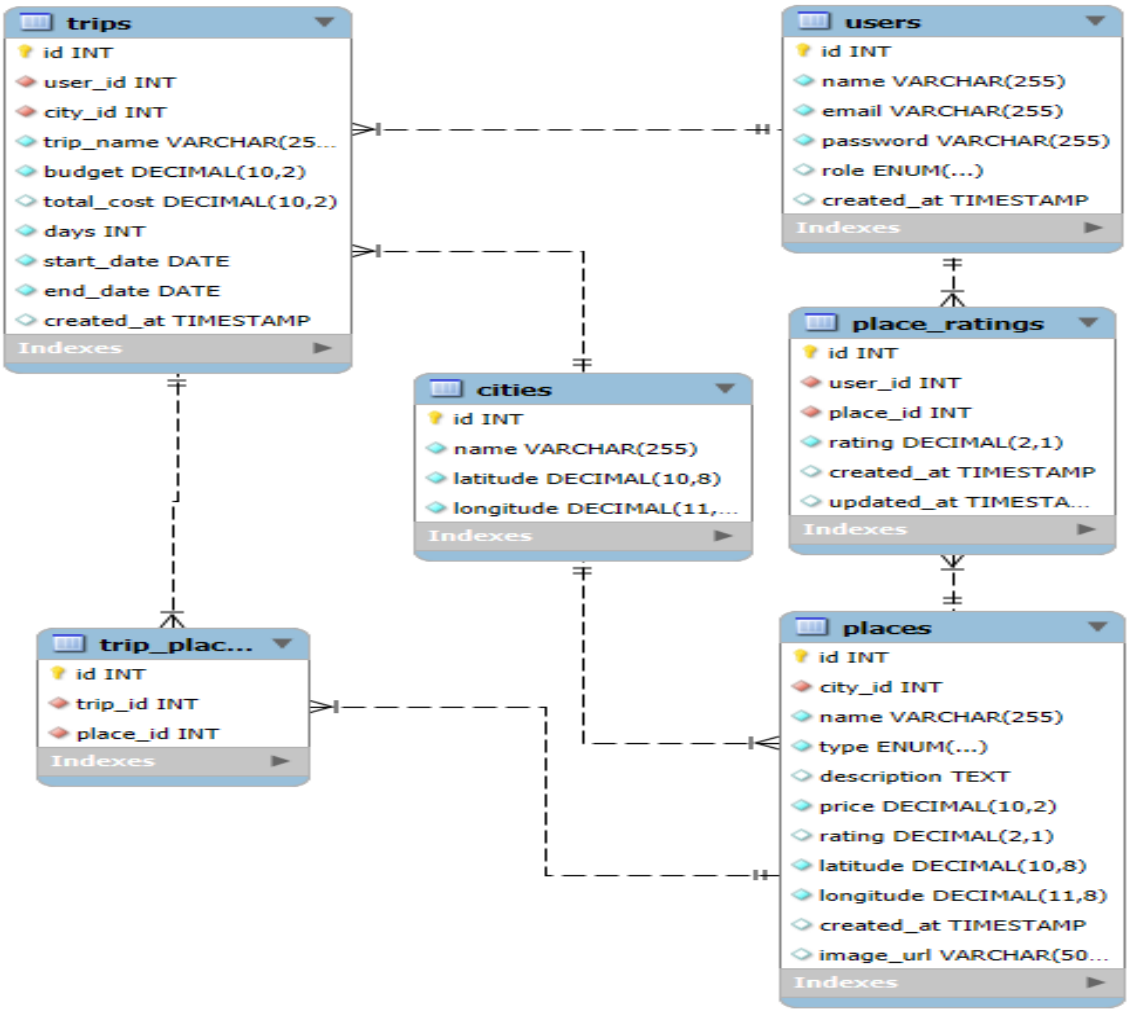



**Figure 3.8 Database Schema Design for Travel and Itinerary Management System**

The database schema of the Travel & Itinerary Management System consists of six main tables: users, cities, places, trips, trip_places, and place_ratings**,** which are interconnected to manage travel data efficiently. The users table stores user information, while cities contain location details. The places table holds data about hotels, restaurants, and attractions and is linked to cities through a one-to-many relationship. The trips table stores user-created itineraries, including budget, duration, and dates, and is connected to users and cities. The trip_places table acts as a junction table linking trips and places, allowing multiple places to be included in a single trip. Additionally, the place_ratings table connects users and places, enabling users to rate locations. Overall, the schema ensures proper data organization, relationships, and efficient retrieval of travel-related information.

### 3.2.5 Wireframing

Wireframing is the process of creating low-fidelity visual representations of a system's user interface. It serves as a blueprint for the design and functionality of an application, helping developers and designers plan the layout and user experience before coding begins.

**Explore Page**

**Figure 3.9 Wire Framing of Explore Page**

**Login Page**

**Figure 3.10 Wire Framing of Login Page**

**Auto-itinerary Generation Page**

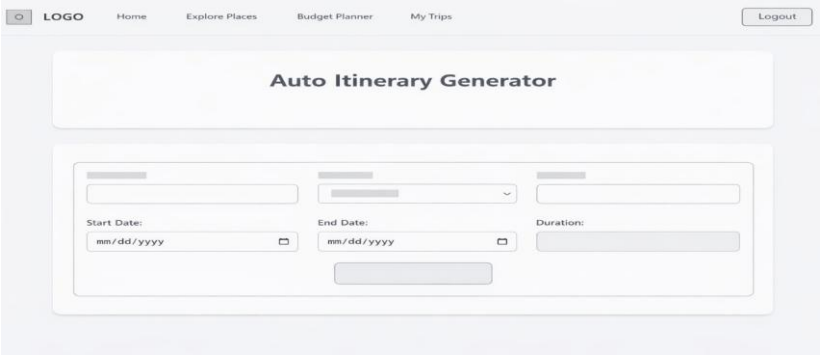



**Figure 3.11 Wire Framing of Auto-itinerary Generation Page**

### 3.2.6 Interface Design

The Interface Design section of software documentation focuses on how users will interact with the system. It includes visual representations and structural diagrams that define the user interface layout, navigation flow, and interaction elements.

**Explore Page**

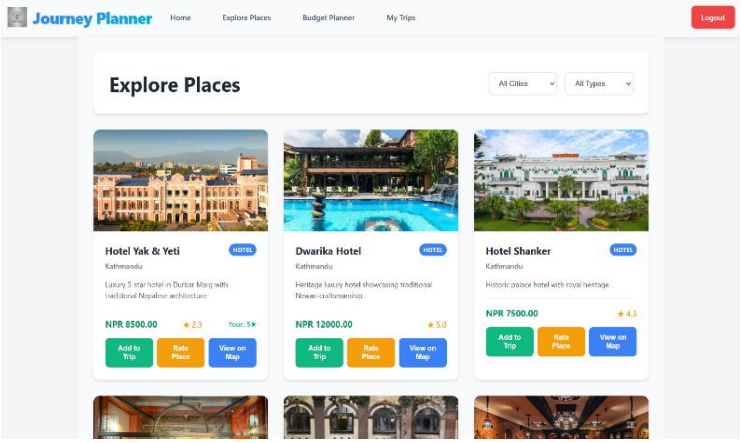



**Figure 3.12 Interface Design of Explore Page**

**Login Page**

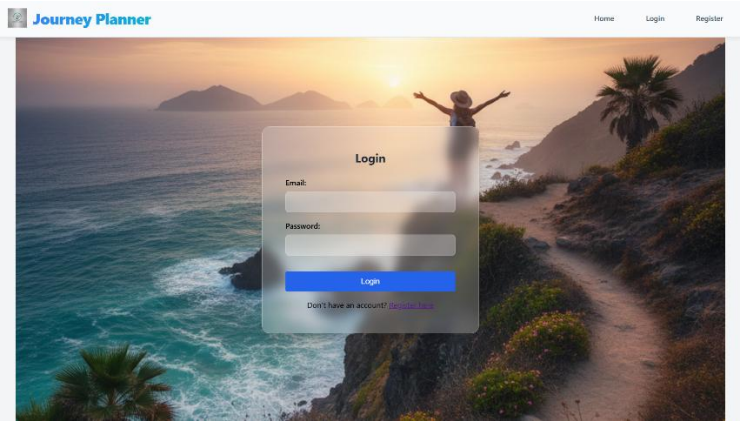



**Figure 3.13 Interface Design of Login Page**

**Auto-itinerary Generation Page**

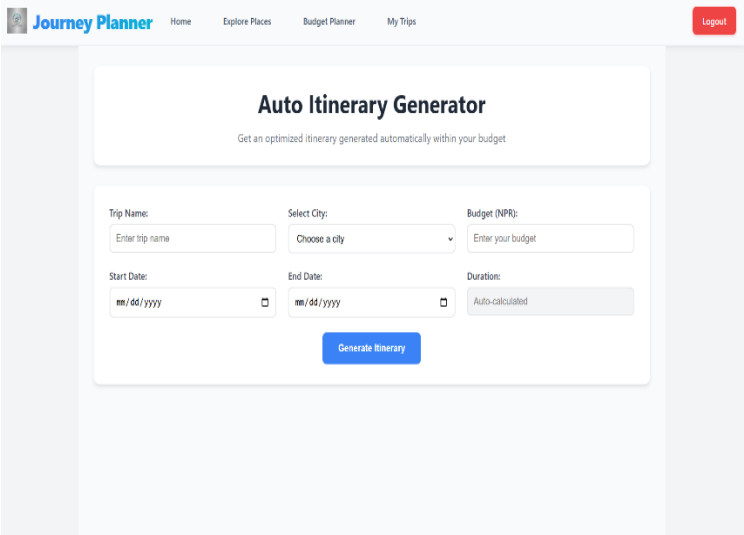



**Figure 3.14 Interface Design of Auto-itinerary Generation Page**

### 3.2.7 Algorithm

**Algorithm Used (0/1 Knapsack Algorithm)**

Knapsack Algorithm is used to optimize itinerary planning by selecting the best combination of places within a given budget. It is used for the initial selection of high-rated places that fit within budget constraints, ensuring quick and efficient choice of essential items such as hotels, restaurants, and attractions. This project uses a budget optimization algorithm based on the 0/1 Knapsack Algorithm combined with Greedy techniques, implemented using JavaScript in the backend. The algorithm is designed to generate an optimized travel itinerary by selecting the best combination of hotels, restaurants, and attractions within a given budget. The system attempts to maximize the overall value (ratings of places) while staying within the user's budget constraints.

**i. Data Preparation**

The system retrieves data from the database including places (hotels, restaurants, attractions), prices, ratings. Places are categorized into hotels, restaurants, attractions. The data is sorted based on ratings (highest first) to prioritize better places. Prices are converted into numeric values to allow calculations.

**ii. Initial Selection using Greedy Approach**

Greedy Algorithm is used for initial selection of high-rated places that fit within constraints. The system first selects one hotel, restaurant, attraction per day and election is based on highest rating which must fit within remaining budget. If high-rated options are not affordable the system selects the cheapest available option.

**iii. Budget Validation**

Minimum Budget = (Cheapest Hotel) + (Cheapest Restaurant × Days) + (Cheapest Attraction × Days). If user budget is less than minimum the system returns an error message. This ensures feasibility before generating itinerary.

**iv. Applying 0/1 Knapsack Algorithm**

After the basic allocation of hotel, restaurants, and attractions, the system calculates the remaining budget. This remaining budget is then used to select additional places using the 0/1 Knapsack Algorithm. Each place is treated as: Weight = Price of place, Value = Rating of place.

**Mathematical Formula**

dp[i][w]=max(value[i]+dp[i−1][w−weight[i]],dp[i−1][w])dp[i][w] = max( value[i] + dp[i-1][w - weight[i]], dp[i-1][w] )dp[i][w]=max(value[i]+dp[i−1][w−weight[i]],dp[i−1][w])

**Implementation**

```
function knapsack(capacity, weights, values, items) {
  const n = items.length;
  // Create DP table
  const dp = Array(n + 1)
    .fill(null)
    .map(() => Array(capacity + 1).fill(0));
  // Build DP table
  for (let i = 1; i <= n; i++) {
    for (let w = 1; w <= capacity; w++) {
      if (weights[i - 1] <= w) {
        // Applying the formula
        dp[i][w] = Math.max(
          values[i - 1] + dp[i - 1][w - weights[i - 1]], // Include item
          dp[i - 1][w] );                                // Exclude item
        } else {
        dp[i][w] = dp[i - 1][w];
      }
}
}
  return dp[n][capacity];
}
```

**v. Round-Robin Distribution**

Additional selected places are distributed across days evenly using (dayIndex = index % totalDays). It ensures balanced itinerary and avoids overloading a single day.

**vi. Rating Calculation (Aggregation)**

Average Rating = SUM(ratings) / COUNT(ratings)

This updated rating is used in future recommendations.

**Example of Working**

Budget = NPR 20,000

Days = 2

Steps:

Step 1: Select 1 hotel, 2 restaurants, 2 attractions

Step 2: Calculate remaining budget

Step 3: Apply Knapsack to add extra places

Step 4: Distribute places across days

Step 5: Generate final itinerary

# Chapter 4: Implementation and Testing

## 4.1 Implementation

Implementation is the phase in system development where the designed system is converted into a working application by coding, integrating components, and deploying all functionalities according to the specified requirements.

### 4.1.1 Tools Used

**Front-End Tools**

**• HTML**

In this project, HTML is used to structure the web pages and define the layout of the application. It provides the basic framework for elements such as navigation bars, forms, buttons, and content sections in the Travel & Itinerary Management System.

**• CSS**

CSS is used to style and design the user interface of the application. It controls the layout, colors, fonts, spacing, and responsiveness of the website. Media queries are implemented to ensure that the system works properly on different screen sizes and devices.

**• JavaScript**

JavaScript is used to add interactivity and dynamic behavior to the frontend. It handles user actions such as form inputs, button clicks, and API calls. It also helps in updating UI components dynamically without reloading the page.

**• React.js**

React.js is used as the main frontend library to build a fast and responsive user interface. It allows the application to be divided into reusable components such as Navbar, Cards, Forms, and Trip Views. React improves performance using a virtual DOM and enables smooth user experience. It also facilitates efficient state management and real-time updates, enhancing overall application responsiveness.

**Back-End Tools**

**• Node.js**

Node.js is used to run JavaScript on the server side. It allows handling multiple requests efficiently and supports building scalable backend services for the application. Node.js lets developers use JavaScript to write command line tools and server-side scripting. The ability to run JavaScript code on the server is often used to generate dynamic web page content before the page is sent to the user's web browser.

**• Express.js**

Express.js is used as a backend framework to handle routing, API creation, and server logic. It processes client requests, communicates with the database, and sends appropriate responses back to the frontend. It also ensures efficient handling of HTTP requests and middleware integration for better application performance and scalability.

**Database Tool**

**• MySQL**

MySQL is used to store and manage all application data. It stores user details, cities, places (hotels, restaurants, attractions), trips, and ratings. It also manages relationships between tables and supports efficient data retrieval and manipulation.

**Development Tools**

**• XAMPP / Local Server (Optional)**

It is used to set up a local development environment for database management and testing. It helps in running MySQL and managing database operations locally.

**• Postman**

Postman is used for testing backend APIs. It helps verify request-response cycles, debug endpoints, and ensure proper functioning of server-side logic.

**• VS Code**

Visual Studio Code is used as the primary code editor for writing, debugging, and managing both frontend and backend code.

**Algorithm & Logic**

**Knapsack Algorithm (Dynamic Programming)**

The 0/1 Knapsack Algorithm is used to optimize itinerary planning by selecting the best combination of places within a given budget while maximizing the overall value, which in this system is based on place ratings. After selecting essential items (hotel, restaurants, and attractions), the remaining budget is calculated. Each place is treated as an item where the price acts as weight and the rating acts as value**.**

**Greedy Algorithm**

The Greedy Algorithm is used in the initial stage of itinerary generation to quickly select essential places such as hotels, restaurants, and attractions. The system sorts all available places based on their ratings in descending order and selects the highest-rated options that fit within the user's budget. This approach provides a fast and efficient way to make initial selections without exploring all possible combinations. If high-rated options are not affordable, the system falls back to selecting the cheapest available options.

### 4.1.2 Implementation Details of Modules

The system's modules are shown below:

- **Create Module**

```
// Add new place with image
router.post('/places', auth, adminAuth, upload.single('image'), async (req, res) => {
  try {
    const { city_id, name, type, description, price, latitude, longitude } = req.body;
    const image_url = req.file ? `/uploads/${req.file.filename}` : null;
    const cleanData = {
      city_id: city_id || null,
      name: name || null,
      type: type || null,
      description: description || null,
      price: price || null,
      latitude: latitude || null,
      longitude: longitude || null,
      image_url
    };
    const [result] = await db.execute(
      'INSERT INTO places (city_id, name, type, description, price, rating, latitude, longitude, image_url) VALUES (?, ?, ?, ?, ?, 0, ?, ?, ?)',
      [cleanData.city_id, cleanData.name, cleanData.type, cleanData.description, cleanData.price, cleanData.latitude, cleanData.longitude, cleanData.image_url]
    );

    res.status(201).json({ id: result.insertId, message: 'Place added successfully' });
  } catch (error) {
    console.error('Backend error:', error);
    res.status(400).json({ message: 'Failed to add place', error: error.message });
  }
});
```

**Listing 4.1 Code Snippet to add new places**

- **Read Module**

```
// Get all places
router.get('/places', auth, adminAuth, async (req, res) => {
  try {
    const [places] = await db.execute(`
      SELECT p.*, c.name as city_name
      FROM places p
      JOIN cities c ON p.city_id = c.id
    `);
    res.json(places);
  } catch (error) {
    res.status(500).json({ message: 'Failed to fetch places', error: error.message });
  }
});
```

**Listing 4.2 Code Snippet to get all places**

- **Update Module**

```
// Update place
router.put('/places/:id', auth, adminAuth, upload.single('image'), async (req, res) => {
  try {
    const { id } = req.params;
    const { city_id, name, type, description, price, latitude, longitude } = req.body;
    const cleanData = {
      city_id: city_id || null,
      name: name || null,
      type: type || null,
      description: description || null,
      price: price || null,
      latitude: latitude || null,
      longitude: longitude || null
    };
    const image_url = req.file ? `/uploads/${req.file.filename}` : null;

    let query, params;

    if (image_url) {
      // Update with new image
      query = 'UPDATE places SET city_id = ?, name = ?, type = ?, description = ?, price = ?, latitude = ?, longitude = ?, image_url = ? WHERE id = ?';
      params = [cleanData.city_id, cleanData.name, cleanData.type, cleanData.description, cleanData.price, cleanData.latitude, cleanData.longitude, image_url, id];
    } else {
      // Update without changing image
      query = 'UPDATE places SET city_id = ?, name = ?, type = ?, description = ?, price = ?, latitude = ?, longitude = ? WHERE id = ?';
      params = [cleanData.city_id, cleanData.name, cleanData.type, cleanData.description, cleanData.price, cleanData.latitude, cleanData.longitude, id];
    }

    await db.execute(query, params);
    res.json({ message: 'Place updated successfully' });
  } catch (error) {
    console.error('Update error:', error);
    res.status(400).json({ message: 'Failed to update place', error: error.message });
  }
});
```

**Listing 4.3 Code Snippet to Update places**

- **Delete Module**

```
// Delete place
router.delete('/places/:id', auth, adminAuth, async (req, res) => {
  try {
    const { id } = req.params;
    await db.execute('DELETE FROM places WHERE id = ?', [id]);
    res.json({ message: 'Place deleted successfully' });
  } catch (error) {
    res.status(400).json({ message: 'Failed to delete place', error: error.message });
  }
});
```

**Listing 4.4 Code Snippet to Delete places**

- **Algorithm Module**

```
// 0/1 Knapsack Algorithm Implementation
function knapsack(capacity, weights, values, items) {
  const n = items.length;
  const dp = Array(n + 1).fill(null).map(() => Array(capacity + 1).fill(0));

  for (let i = 1; i <= n; i++) {
    for (let w = 1; w <= capacity; w++) {
      if (weights[i - 1] <= w) {
        dp[i][w] = Math.max(
          values[i - 1] + dp[i - 1][w - weights[i - 1]],
          dp[i - 1][w]
        );
      } else {
        dp[i][w] = dp[i - 1][w];
      }
    }
  }
```

**Listing 4.5 Code Snippet of Algorithm Implementation**

## 4.2 Testing

System Testing is done by giving different training and testing datasets. This test is done to evaluate whether the system is providing accurate summary or not. During the phase of the development of the system, our system is tested time and again. The series of testing conducted are as follows:

### 4.2.1 Test Cases for Unit Testing

In unit testing, we designed the entire system in modularized pattern and each module is tested. Until we get the accurate output from the individual module. The input forms are tested so that they do not accept invalid input.

**Table 4.1 Test Case for Login**

| Test Id | Test Case Description | Test Input | Expected Output | Actual Output | Remark |
|---|---|---|---|---|---|
| 1. | Open Login Page | http://localhost/login | Displays login page | Displays login page | Pass |
| 2. | Login with empty fields | Email: — Password: — | Validation error shown | Validation error shown | Pass |
| 3. | Login with invalid email format | Email: usergmail.com | Enter Password error | Invalid password error | Pass |
| 4. | Login with wrong password | Email: user@gmail.com Password: wrong123 | Invalid Password | Invalid Password | Pass |
| 5. | Login with correct credentials | Email= user@gmail.com Password= 123456789 | Redirect to user dashboard | Redirect to user dashboard | Pass |

**Table 4.2 Test Case for Itinerary Generation**

| Test Id | Test Case Description | Test Input | Expected Output | Actual Output | Remark |
|---|---|---|---|---|---|
| 1. | Open itinerary generator | Click "Plan Trip" | Form is displayed | Form is displayed | Pass |
| 2. | Submit empty trip form | No input | Validation error shown | Validation error shown | Pass |
| 3. | Enter valid trip details | City: KTM, Budget: 10000, Days: 2 | Itinerary generated | Itinerary generated | Pass |
| 4. | Budget too low | Budget: 500 | Error message (minimum budget) | Error displayed | Pass |
| 5. | Generate itinerary within budget | Budget: 15000 | Optimized plan shown | Optimized plan shown | Pass |

**Table 4.3 Test Case for Admin Pannel**

| Test Id | Test Case Description | Test Input | Expected Output | Actual Output | Remark |
|---|---|---|---|---|---|
| 1. | Admin login | Email: admin@mail.com Password: 123456 | Redirect to admin dashboard | Redirect to admin dashboard | Pass |
| 2. | Add new place | Name: Hotel ABC | Place added successfully | Place added | Pass |
| 3. | Update place details | Change price/rating | Updated successfully | Updated successfully | Pass |
| 4. | Delete place | Select place = Delete | Place removed | Place removed | Pass |
| 5. | View all users | Click "Users" | User list displayed | User list displayed | Pass |

### 4.2.2 Test Cases for System Testing

In system testing, whole system is tested as below:

**Table 4.4 System Testing for Travel and Itinerary Management System**

| Test Id | Objectives | Test Cases | Expected Outcomes | Test Result |
|---|---|---|---|---|
| 1. | Verify user registration | Click on Register and submit valid details | User account is created successfully | Pass |
| 2. | Verify user login | Enter valid credentials and click Login | User is logged into the system and redirected to dashboard | Pass |
| 3. | Validate place exploration | Click on Explore Places | List of hotels, restaurants, and attractions is displayed | Pass |
| 4. | Test adding places to trip | Click on Add to Trip | Selected place is added to the user's trip | Pass |
| 5. | Verify budget planner form | Click on Budget Planner | Budget input form is displayed | Pass |
| 6. | Validate itinerary generation | Enter budget and days, click Generate | Optimized itinerary is generated within budget | Pass |
| 7. | Verify trip viewing | Click on View My Trips | List of saved trips is displayed | Pass |
| 8. | Test trip deletion | Click on Delete Trip | Selected trip is deleted successfully | Pass |
| 9. | Validate place rating | Submit rating for a place | Rating is stored and updated successfully | Pass |
| 10 | Test trip editing | Click on Edit Trip and update details | Trip is updated successfully | Pass |

## 4.3 Result and Analysis

The Travel & Itinerary Management System has been successfully developed and implemented as a web-based platform that enables users to plan and manage their trips efficiently. The system allows users to register and authenticate securely, ensuring personalized access to travel planning features. Users can explore curated data of hotels, restaurants, and tourist attractions across selected cities such as Kathmandu, Pokhara, and Biratnagar. The platform provides an interactive interface where users can create itineraries, estimate travel costs, and receive optimized recommendations based on their budget using the 0/1 Knapsack algorithm.

A thorough evaluation of the system highlights several strengths, including its user-friendly interface, structured data management, and the integration of an optimization algorithm for budget-based recommendations. The use of a centralized database ensures data integrity and efficient retrieval of information. The system simplifies complex travel planning tasks by combining multiple functionalities such as place exploration, itinerary generation, and cost estimation into a single platform. Furthermore, the use of map integration enhances user experience by providing geographical visualization of locations, making trip planning more intuitive and realistic.

However, certain limitations still exist within the system. Currently, the application is restricted to a limited number of cities, which reduces its scalability in terms of geographical coverage. Additionally, while the system provides estimated budgeting, real-time pricing and availability of services are not integrated. Some features, such as saving trips permanently in the database or real-time collaboration among users, can be further improved. User engagement may also be limited without advanced features such as personalized recommendations or reviews for better decision-making.

To enhance the system further, several improvements can be implemented in future versions. These include expanding the system to support more cities and destinations, integrating real-time data such as hotel availability and pricing, and implementing secure online booking features. Additional enhancements such as a mobile application, user review and rating system for better recommendations, and AI-based personalized travel suggestions can significantly improve usability.

Furthermore, improving system scalability and performance will be essential for handling a larger user base. Integration with external APIs such as maps and booking services can make the platform more dynamic and practical for real-world usage. By addressing these challenges and incorporating advanced features, the system has the potential to evolve into a comprehensive travel planning solution.

Despite these limitations, the Travel & Itinerary Management System successfully achieves its primary objective of simplifying travel planning and itinerary management. It provides a structured, efficient, and user-friendly platform that reduces manual effort and enhances decision-making. With continuous improvements and feature enhancements, the system has strong potential to become a reliable and scalable travel planning solution.

# Chapter 5: Conclusion and Future Recommendations

## 5.1 Lesson Learnt

Every project provides an opportunity to learn and gain knowledge in various aspects. Through the development of the Travel & Itinerary Management System, we have enhanced our problem-solving abilities and developed important skills such as teamwork, independent thinking, effective communication, and project management.

**• Project Management**

Through this project, we learned how to plan and execute a system from requirement analysis to final implementation. We improved our time management skills by dividing tasks into phases such as design, development, testing, and documentation, ensuring timely completion of the project.

**• Enhanced Technical Skills**

Working on this project significantly improved our technical knowledge in both frontend and backend development. We gained hands-on experience with HTML, CSS, JavaScript, and React for building interactive user interfaces. On the backend, we developed skills in Node.js, Express, and MySQL for handling server-side logic, API development, and database management. Additionally, implementing the Knapsack algorithm helped us understand real-world application of dynamic programming.

**• Teamwork**

As this was a group project, we learned how to collaborate effectively by sharing responsibilities, communicating ideas, resolving conflicts, and supporting each other throughout the development process. We also improved our ability to coordinate tasks efficiently and maintain consistency in development through proper teamwork and communication.

**• Problem-Solving Skills**

During development, we faced challenges such as algorithm implementation, data handling, and system integration. Solving these issues improved our analytical thinking and helped us develop efficient solutions. It also enhanced our ability to debug errors, optimize performance, and approach problems in a structured and logical manner.

**• Documentation Skills**

We gained experience in preparing technical documentation, including diagrams, reports, and structured explanations. This improved our writing skills and familiarity with tools like MS Word and diagramming software.

## 5.2 Conclusion

The Travel & Itinerary Management System is designed to simplify and enhance the process of travel planning by providing a structured and user-friendly platform. Through proper research, planning, and system design, we developed a web application that allows users to explore places, generate itineraries, and estimate travel costs efficiently. The system integrates features such as map visualization, admin-controlled data management, and budget optimization using the 0/1 Knapsack algorithm. Users can plan trips based on their preferences and budget, while administrators can manage cities, places, and system data effectively. Although the system currently supports limited cities and features, it successfully demonstrates how technology can streamline travel planning. With further improvements, it has the potential to become a more advanced and practical travel solution.

## 5.3 Future Recommendation

The future success of this system depends on continuous improvement and user feedback. The system can be enhanced by expanding its features, improving scalability, and integrating real-world services. Some of the future recommendations for this system are:

- Expanding the system to include more cities and destinations
- Implementing real-time booking for hotels and restaurants

# References


[1] GetYourGuide, "GetYourGuide," :GetYourGuide Deutschland GmbH, 2008. [Online]. Available: https://www.getyourguide.com/. [Accessed 2025].

[2] Z. Z. Travels, "Zip Zap Travels," Zip Zap Travels, [Online]. Available: https://zipzaptravels.com/. [Accessed 2025].

[3] N. T. P. P. Ltd., "Nepal Trip Planner," Nepal Trip Planner Pvt. Ltd., [Online]. Available: https://www.nepaltripplanners.com/. [Accessed 2025].

[4] Tripadvisor, "Tripadvisor," Tripadvisor, Inc. , 2000. [Online]. Available: https://www.tripadvisor.com/. [Accessed 2025 ].

[5] N. T. P. P. Ltd., "Nepal Trekking Planner," Nepal Trekking Planner Pvt. Ltd., 2000. [Online]. Available: https://www.nepaltrekkingplanner.com/. [Accessed 2025].

[6] Viator, "Viator," Tripadvisor, Inc., [Online]. Available: https://www.viator.com/. [Accessed 2025].

[7] Musement, "Musement," TUI Group, 2013. [Online]. Available: https://www.musement.com/. [Accessed 2025].

[8] Trekiz, "Trekiz pvt ltd," Trekiz pvt ltd, [Online]. Available: http://www.trekiz.com/. [Accessed 2025].

[9] A. N. T. Trekking, "Access Nepal Tour & Trekking," Access Nepal Tour & Trekking Pvt. Ltd., [Online]. Available: https://www.accessnepaltour.com/. [Accessed 2025].

[10] T. Local, "Travel Local," Travel Local Ltd., [Online]. Available: https://www.travellocal.com/. [Accessed 2025].

[11] M. J. Abu-AlShaeer, "A Comprehensive Online Tourism Management System Revolutionizes Travel," FRUCT Conference Proceedings, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10560169. [Accessed 2025].

[12] Z. Xiang, "Adapting to the Internet: Trends in Travelers Use of the Web for Trip Planning," Journal of Travel Research (SAGE Publishing), 2015. [Online]. Available: https://doi.org/10.1177/0047287514522883. [Accessed 2025].

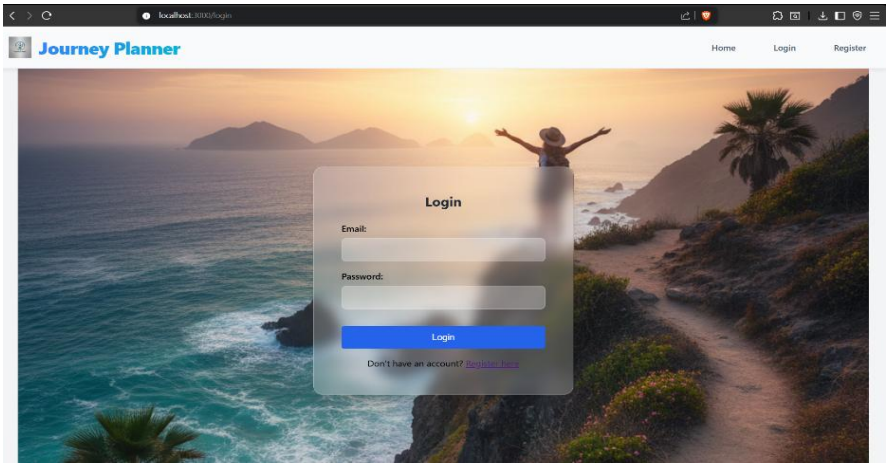
[13] W. L. Chen, "Constructing a Personalized Travel Itinerary Recommender System with the Internet of Things," Wireless Networks (Springer), 2023. [Online]. Available: https://doi.org/10.1007/s11276-023-03453-y. [Accessed 2025].

[14] A. Maruyama, "A Personal Tourism Navigation System to Support Traveling Multiple Destinations with Time Restrictions," AINA 2004 Conference Proceedings, 2004. [Online]. Available: https://doi.org/10.1109/AINA.2004.1283747. [Accessed 2025].

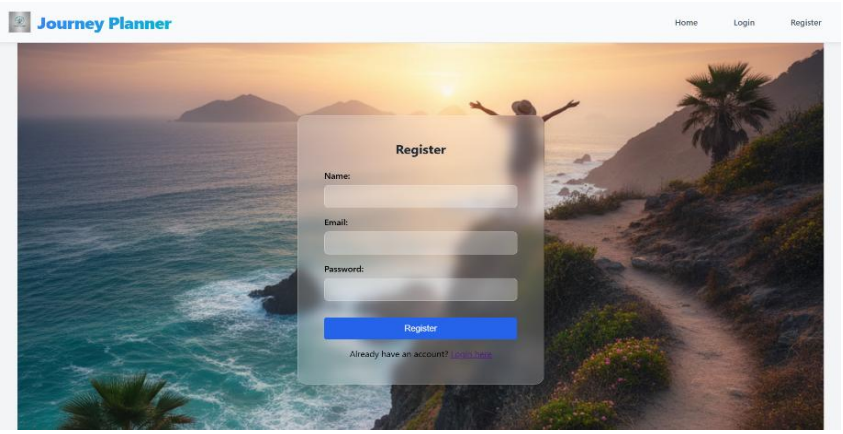
[15] M. A. Jan, "Smart Tourist Route Recommendation Algorithm," Wireless Communications, 2022. [Online]. Available: https://onlinelibrary.wiley.com. [Accessed 2025].

# Appendices

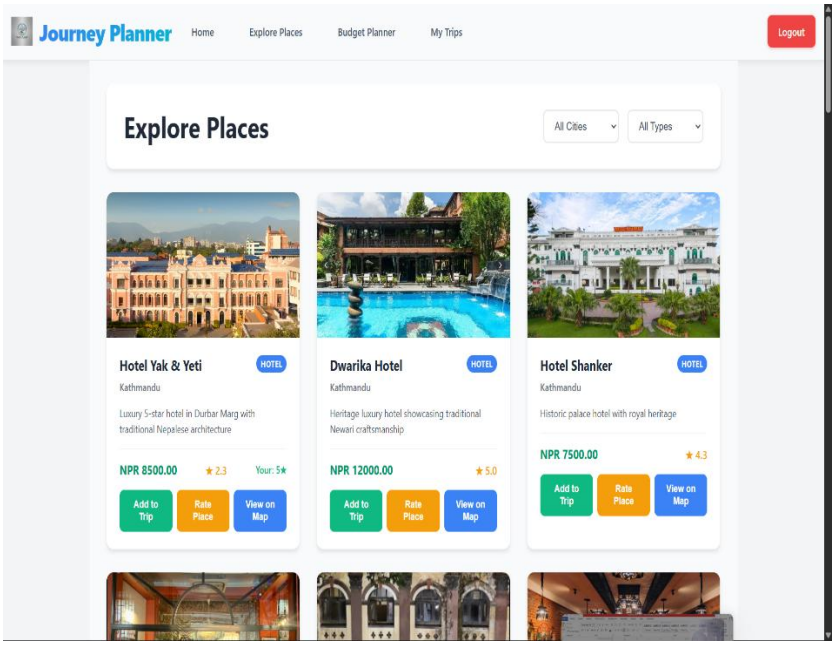
- **Login page**



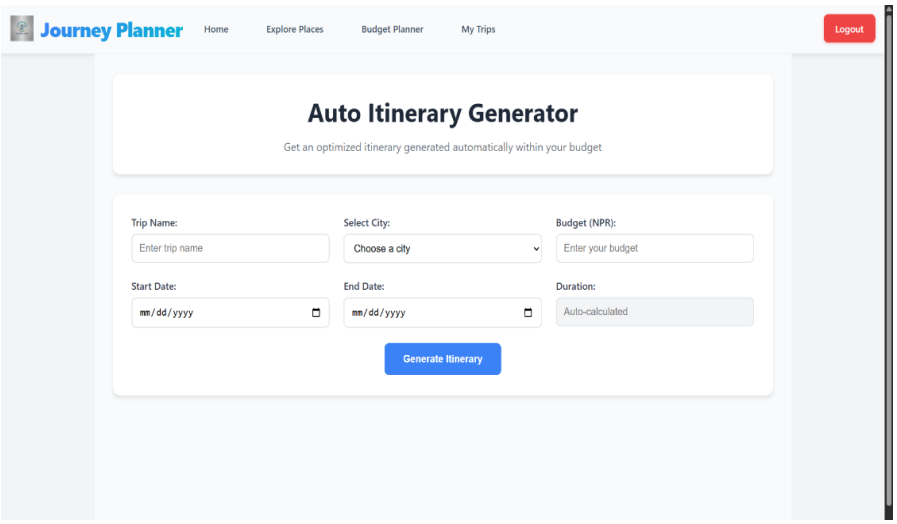

- **Register page**



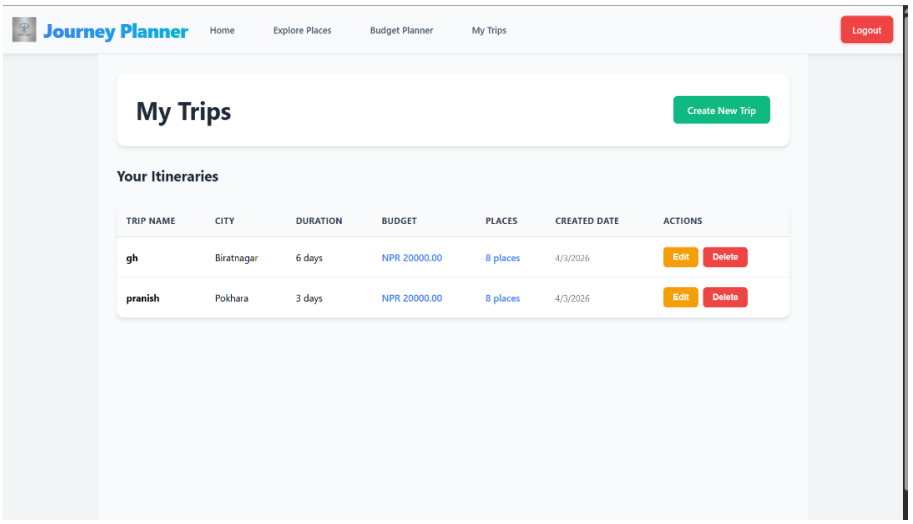

- **Explore Places**



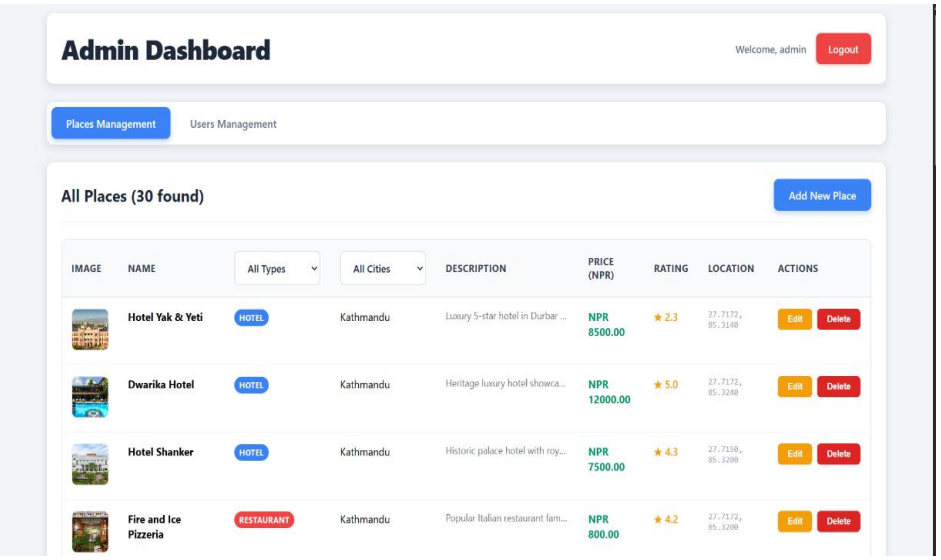
- **Auto Itinerary Generator page**




- **User Detail page**




- **Admin Dashboard page**



- **CRUD code**

```
const express = require('express');
const db = require('../config/database');
const { auth } = require('../middleware/auth');
const router = express.Router();
router.post('/', auth, async (req, res) => {
try {
const { tripName, cityId, days, budget, startDate, endDate, places, totalCost } = req.body;
const userId = req.user.id;
const values = [userId, parseInt(cityId), String(tripName), parseFloat(budget),
parseFloat(totalCost) || 0, parseInt(days),
String(startDate).split('T')[0], String(endDate).split('T')[0]];
const [tripResult] = await db.execute(
'INSERT INTO trips (user_id, city_id, trip_name, budget, total_cost, days, start_date,
end_date, created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, NOW())',values);
const tripId = tripResult.insertId;
if (places && places.length > 0) {
const uniquePlaceIds = [...new Set(places.map(p => p.id).filter(id => id != null))];
for (const placeId of uniquePlaceIds) {
await db.execute('INSERT INTO trip_places (trip_id, place_id) VALUES (?, ?)', [tripId,
parseInt(placeId)]);
}    }
res.status(201).json({ success: true, tripId, message: 'Trip saved successfully' });
} catch (error) {
res.status(500).json({ success: false, message: 'Failed to create trip', error: error.message
});
}});
router.get('/', auth, async (req, res) => {
try {
const userId = req.user.id;
const [tripsData] = await db.execute(`
SELECT t.id, t.trip_name, t.budget, t.total_cost, t.days, t.start_date, t.end_date,
t.created_at,c.name as city_name, c.id as city_id, COUNT(tp.id) as places_count
FROM trips t
```

```
JOIN cities c ON t.city_id = c.id
LEFT JOIN trip_places tp ON t.id = tp.trip_id
WHERE t.user_id = ?
GROUP BY t.id
ORDER BY t.created_at DESC`, [userId]);
const tripsWithPlaces = await Promise.all(tripsData.map(async (trip) => {
const [places] = await db.execute(`
SELECT p.* FROM places p
JOIN trip_places tp ON p.id = tp.place_id
WHERE tp.trip_id = ?`, [trip.id]);
return { ...trip, places };
}));
res.json({ success: true, data: tripsWithPlaces });
} catch (error) {
res.status(500).json({ success: false, message: 'Failed to fetch trips' });
}});
router.delete('/:id', auth, async (req, res) => {
try {
const { id } = req.params;
const userId = req.user.id;
const [trip] = await db.execute('SELECT id FROM trips WHERE id = ? AND user_id =
?', [id, userId]);
if (trip.length === 0) return res.status(404).json({ success: false, message: 'Trip not found'
});
await db.execute('DELETE FROM trips WHERE id = ? AND user_id = ?', [id, userId]);
res.json({ success: true, message: 'Trip deleted successfully' });
} catch (error) {
res.status(500).json({ success: false, message: 'Failed to delete trip' });
}});
module.exports = router;
```

- Algorithm Code

```
const express = require('express');
```

```
const db = require('../config/database');
const router = express.Router();
function knapsack(capacity, weights, values, items) {
const n = items.length;
const dp = Array(n + 1).fill(null).map(() => Array(capacity + 1).fill(0));
for (let i = 1; i <= n; i++) {
for (let w = 1; w <= capacity; w++) {
if (weights[i - 1] <= w) {
dp[i][w] = Math.max(
values[i - 1] + dp[i - 1][w - weights[i - 1]],
dp[i - 1][w]);
} else {
dp[i][w] = dp[i - 1][w];
}}}
const selectedItems = [];
let w = capacity;
for (let i = n; i > 0 && dp[i][w] > 0; i--) {
if (dp[i][w] !== dp[i - 1][w]) {
selectedItems.push(items[i - 1]);
w -= weights[i - 1];
}}
return { maxValue: dp[n][capacity], selectedItems: selectedItems.reverse() };
}
router.post('/budget', async (req, res) => {
try {
const { city_id, budget, days, hotelPreference, restaurantPreference } = req.body;
const [places] = await db.execute('SELECT * FROM places WHERE city_id = ?',
[city_id]);
const hotels = places.filter(p => p.type === 'HOTEL').sort((a, b) => b.rating - a.rating);
const restaurants = places.filter(p => p.type === 'RESTAURANT').sort((a, b) => b.rating
- a.rating);
const attractions = places.filter(p => p.type === 'ATTRACTION').sort((a, b) => b.rating -
a.rating);
const applyPreference = (list, preference) => {
```

```
const sorted = [...list];
if (preference === 'budget') return sorted.sort((a, b) => parseFloat(a.price) -
parseFloat(b.price));
if (preference === 'luxury') return sorted.sort((a, b) => parseFloat(b.price) -
parseFloat(a.price));
if (preference === 'mid') {
const prices = sorted.map(p => parseFloat(p.price));
const mid = (Math.min(...prices) + Math.max(...prices)) / 2;
return sorted.sort((a, b) => Math.abs(parseFloat(a.price) - mid) -
Math.abs(parseFloat(b.price) - mid));
}
return sorted;
};
const filterByPreference = (list, preference) => {
const prices = list.map(p => parseFloat(p.price));
const min = Math.min(...prices), max = Math.max(...prices), range = max - min;
if (preference === 'budget') return list.filter(p => parseFloat(p.price) <= min + range *
0.33);
if (preference === 'luxury') return list.filter(p => parseFloat(p.price) >= min + range *
0.67);
if (preference === 'mid') return list.filter(p => parseFloat(p.price) > min + range * 0.33
&& parseFloat(p.price) < min + range * 0.67);
return list;
};
const hotelPref = hotelPreference || 'any';
const restaurantPref = restaurantPreference || 'any';
const filteredHotels = filterByPreference(hotels, hotelPref);
const filteredRestaurants = filterByPreference(restaurants, restaurantPref);
const sortedHotels = applyPreference(filteredHotels.length > 0 ? filteredHotels : hotels,
hotelPref);
const sortedRestaurants = applyPreference(filteredRestaurants.length > 0 ?
filteredRestaurants : restaurants, restaurantPref);
const cheapestHotel = Math.min(...hotels.map(h => parseFloat(h.price)));
const cheapestRestaurant = Math.min(...restaurants.map(r => parseFloat(r.price)));
```

```
const cheapestAttraction = Math.min(...attractions.map(a => parseFloat(a.price)));
let selectedHotel = null;
for (const hotel of sortedHotels) {
const minOtherCosts = (cheapestRestaurant * days) + (cheapestAttraction * days);
if ((parseFloat(hotel.price) * days) + minOtherCosts <= budget) {
selectedHotel = hotel;
break;
}}
const hotelCostPerDay = parseFloat(selectedHotel.price);
const budgetPerDay = (budget - (hotelCostPerDay * days)) / days;
const itineraryByDays = [];
let totalCost = hotelCostPerDay * days;
for (let day = 0; day < days; day++) {
let dayBudget = budgetPerDay;
let dayPlaces = [];
const usedRestaurants = itineraryByDays.flatMap(d => d.places).filter(p => p.type ===
'RESTAURANT');
const usedAttractions = itineraryByDays.flatMap(d => d.places).filter(p => p.type ===
'ATTRACTION');
dayPlaces.push({...selectedHotel, day: day + 1});
let restaurantsAdded = 0;
for (const restaurant of sortedRestaurants) {
if (restaurantsAdded >= 2) break;
const price = parseFloat(restaurant.price);
if (!usedRestaurants.find(p => p.id === restaurant.id) && price <= dayBudget &&
(totalCost + price) <= budget) {
dayPlaces.push({...restaurant, day: day + 1});
dayBudget -= price; totalCost += price; restaurantsAdded++;
}}
let attractionsAdded = 0;
for (const attraction of attractions) {
if (attractionsAdded >= 3) break;
const price = parseFloat(attraction.price);
if (!usedAttractions.find(p => p.id === attraction.id) && price <= dayBudget &&
```

```
(totalCost + price) <= budget) {
dayPlaces.push({...attraction, day: day + 1});
dayBudget -= price; totalCost += price; attractionsAdded++;
}}
itineraryByDays.push({ day: day + 1, places: dayPlaces, remainingBudget: dayBudget });
}
const totalRemainingBudget = itineraryByDays.reduce((sum, day) => sum +
day.remainingBudget, 0);
if (totalRemainingBudget > 0) {
const allUsedPlaces = itineraryByDays.flatMap(d => d.places);
const remainingPlaces = places.filter(p =>
p.type !== 'HOTEL' && !allUsedPlaces.find(item => item.id === p.id) &&
parseFloat(p.price) <= totalRemainingBudget
);
if (remainingPlaces.length > 0) {
const weights = remainingPlaces.map(p => Math.ceil(parseFloat(p.price)));
const values = remainingPlaces.map(p => Math.ceil(parseFloat(p.rating) * 10));
const result = knapsack(Math.floor(totalRemainingBudget), weights, values,
remainingPlaces);
result.selectedItems.forEach((item, index) => {
const dayIndex = index % days;
const dayPlaces = itineraryByDays[dayIndex].places;
if (item.type === 'RESTAURANT' && dayPlaces.filter(p => p.type ===
'RESTAURANT').length >= 2) return;
if (item.type === 'ATTRACTION' && dayPlaces.filter(p => p.type ===
'ATTRACTION').length >= 3) return;
if (totalCost + parseFloat(item.price) > budget) return;
itineraryByDays[dayIndex].places.push({...item, day: dayIndex + 1});
totalCost += parseFloat(item.price);
});}}
itineraryByDays.forEach(day => delete day.remainingBudget);
res.json({
itinerary: itineraryByDays,
totalCost: totalCost.toFixed(2),
```

```
remainingBudget: (budget - totalCost).toFixed(2)
});
} catch (error) {
res.status(500).json({ message: 'Failed to generate recommendations', error:
error.message });
}});
module.exports = router;
```